Templates:

Templates are powerful features of C++ which allows you to write generic programs .In simple terms, you can create a single function or a class to work with different data types using templates.

Templates are often used in larger codebase for the purpose of code reusability and flexibility of the programs.

The concept of templates can be used in two different ways:

- Function Templates
- Class Templates

Function Templates

A function template works in a similar to a normal function, with one key difference.

A single function template can work with different data types at once but, a single normal function can only work with one set of data types.

Syntax :

A function template starts with the keyword **template** followed by template parameter/s inside **< >** which is followed by function declaration.

```
template <class T>
T FunctionName(T arg)
{
   ... .. ...
}
```

In the above code, *T* is a template argument that accepts different data types (int, float), and **class** is a keyword.

You can also use keyword **typename** instead of class in the above example.

When, an argument of a data type is passed to `Function( )`, compiler generates a new version of `Function()` for the given data type.

Example : Program to print the biggest of two values

```
#include <iostream>
```

Vector India

```
using namespace std;

// One function works for all data types. This would work
// even for user defined types if operator '>' is overloaded
template <typename T>
T myMax(T x, T y)
{
     return (x > y)? x: y;
}

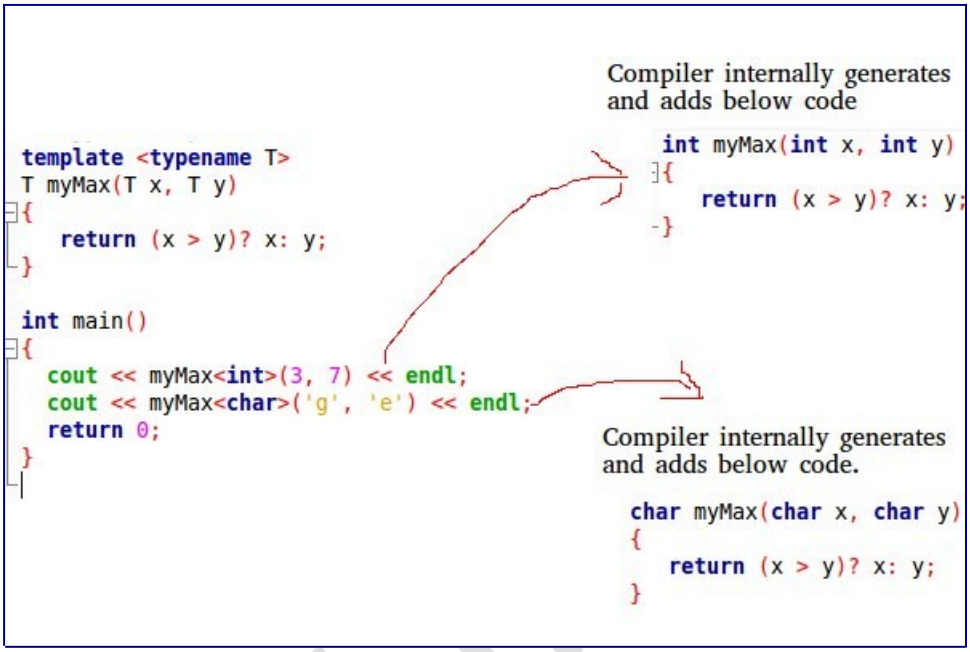
int main()
{
     cout << myMax(3, 7) << endl; // Call myMax for int
     cout << myMax(3.0, 7.0) << endl; // call myMax for double
     cout << myMax('g', 'e') << endl; // call myMax for char
     return 0;
}
```

How templates work?

Templates are expanded at compiler time. This is like macros. The difference is, compiler does type checking before template expansion. The idea is simple, source code contains only function/class, but compiled code may

Vector India

contain multiple copies of same function/class.



Below is the program to implement Bubble Sort using templates :

```
#include <iostream>
using namespace std;
// A template function to implement bubble sort.
// We can use this for any data type that supports
// comparison operator < and swap works for it.
template <class T>
void bubbleSort(T a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = n - 1; i < j; j--)
            if (a[j] < a[j - 1])
                swap(a[j], a[j - 1]);
}


int main() {
int a[5] = {10, 50, 30, 40, 20};
int n = sizeof(a) / sizeof(a[0]);
// calls template function
bubbleSort(a, 5);
cout << " Sorted array : ";
```

```
for (int i = 0; i < n; i++)
     cout << a[i] << " ";
cout << endl;
return 0;
}
Output: 10 20 30 40 50
```

Can there be more than one arguments to templates?Yes, like normal parameters, we can pass more than one data types as arguments to templates.

The following example demonstrates the same.

```
#include<iostream>

using namespace std;

template<class type1,class type2>

void print(type1 a,type2 b)

{

     cout<<a<<" "<<b<<endl;

}

main()

{    int num=123;

     double d=34.5;

     char ch='A';

     string s="vector";

     print(num,d);  // for this function call type1 is treated as integer and type2 is double

     print(ch,s); // for this function call type1 is char and type is string
```

Vector India

```
}
```

Output:

```
123 34.5

A    vector
```

Class Template:

Class Templates Like function templates, class templates are useful when a class defines something that is independent of the data type. Can be useful for classes like LinkedList, BinaryTree, Stack, Queue, Array, etc.

Synatx for class template:

```
template <class T>
class className
{
   ... .. ...
public:
   T var;
   T someOperation(T arg);
   ... .. ...
};
```

In the above declaration, `T` is the template argument which is a placeholder for the data type used.

Inside the class body, a member variable *var* and a member function `someOperation()` are both of type `T`.

Syntax to create a class template object

To create a class template object, you need to define the data type inside a < > when creation.

```
className<dataType> classObject;
```

Example :

```
className<int> classObject;
```


Vector India

```
className<float> classObject;
className<string> classObject;
```

Simple calculator using Class template

Program to add, subtract, multiply and divide two numbers using class template

```
#include <iostream>
using namespace std;

template <class T>
class Calculator
{
private:
        T num1, num2;

public:
        Calculator(T n1, T n2)
        {
                num1 = n1;
                num2 = n2;
        }

        void displayResult()
        {
                cout << "Numbers are: " << num1 << " and " <<
num2 << "." << endl;
                cout << "Addition is: " << add() << endl;
                cout << "Subtraction is: " << subtract() << endl;
                cout << "Product is: " << multiply() << endl;
                cout << "Division is: " << divide() << endl;
        }

        T add() { return num1 + num2; }

        T subtract() { return num1 - num2; }

        T multiply() { return num1 * num2; }

        T divide() { return num1 / num2; }
};
```

Vector India

```
int main()
{
        Calculator<int> intCalc(2, 1);
        Calculator<float> floatCalc(2.4, 1.2);

        cout << "Int results:" << endl;
        intCalc.displayResult();

        cout << endl << "Float results:" << endl;
        floatCalc.displayResult();

        return 0;
}
```

Output

```
Int results:
Numbers are: 2 and 1.
Addition is: 3
Subtraction is: 1
Product is: 2
Division is: 2

Float results:
Numbers are: 2.4 and 1.2
Addition is: 3.6
Subtraction is: 1.2
Product is: 2.88
Division is: 2
```

In the above program, a class template `Calculator` is declared.

The class contains two private members of type `T`: *num1* & *num2*, and a constructor to initalize the members.

It also contains public member functions to calculate the addition, subtraction, multiplication and division of the numbers which return the value of data type defined by the user. Likewise, a function `displayResult()` to display the final output to the screen.

In the `main()` function, two different `Calculator` objects `intCalc` and `floatCalc` are created for data types: `int` and `float` respectively. The values are initialized using the constructor.

Vector India

Notice we use `<int>` and `<float>` while creating the objects. These tell the compiler the data type used for the class creation.

This creates a class definition each for `int` and `float`, which are then used accordingly.

Then, `displayResult()` of both objects is called which performs the Calculator operations and displays the output.

Vector India